

Jaiden Forrest Logan
6/27/2026
CS 320

CS 320 Project Two: Summary and Reflections Report

Summary

For Project One, my testing approach was to check each requirement directly. The project included the contact, task, and appointment services. Since there was no user interface or database, unit testing with JUnit made the most sense because JUnit supports repeatable automated tests for Java code (JUnit Team, n.d.). My goal was to make sure each object class followed its rules and each service could add, update, or delete records correctly.

For the contact feature, I tested the Contact class rules for contact ID, first name, last name, phone number, and address. I tested valid contacts, but I also tested invalid inputs. One test I used to check that an invalid phone number was rejected was:

```
assertThrows(IllegalArgumentException.class, () -> new Contact("12345", "John", "Smith",  
"12345abcde", "123 Main Street"));
```

For ContactService, I tested adding contacts, blocking duplicate IDs, deleting contacts, and updating the allowed fields by contact ID. For the task feature, I tested the task ID, name, and description requirements. One test checked that a task ID longer than 10 characters would fail:

```
assertThrows(IllegalArgumentException.class, () -> new Task("12345678901", "Finish  
report", "Complete the weekly report"));
```

For TaskService, I tested adding tasks, blocking duplicate task IDs, deleting tasks, and updating the name and description. For the appointment feature, I tested the appointment ID, appointment date, and description rules. I tested future dates and past dates because appointment dates could not be in the past. For AppointmentService, I tested adding appointments, blocking duplicate IDs, deleting appointments, and trying to retrieve appointments that did not exist.

Overall, my approach matched the requirements because the tests were based on the rules in the directions. I tested valid and invalid cases instead of only testing examples that worked. For example, I tested invalid phone numbers because the phone number had to be exactly 10 digits. I

tested past appointment dates because appointments could not be in the past. I also kept the ID fields final and did not include setter methods because IDs were not supposed to be updatable. I think the tests were effective because all three services passed and coverage was above the required amount. The final coverage results were around 83.4% for ContactService, 87.9% for TaskService, and 87.5% for AppointmentService. Coverage does not prove the tests are perfect, but it does show that most of the code was reached and that the main validation and service methods were tested.

Writing the JUnit tests made me think more carefully about bad input. Null values, fields that were too long, invalid phone numbers, duplicate IDs, and past appointment dates all needed to be tested because those are the kinds of problems that can break requirements. I made sure the code was technically sound by using assertions that matched the expected result. For example, this checked that the service returned the correct task name:

```
assertEquals("Finish report", service.getTask("10001").getName());
```

I made the code efficient by keeping the services simple and using in-memory maps keyed by ID. This made add, find, update, and delete operations direct instead of requiring unnecessary loops. I also kept the test setup simple. For example, this checked that adding one task changed the count correctly:

```
assertEquals(1, service.getTaskCount());
```

Reflection

The main testing technique I used was unit testing. Unit testing checks small pieces of code by themselves. In this project, that meant testing each object class and service class separately. I also used positive and negative testing. Positive tests checked that valid input worked, while negative tests checked that invalid input was rejected. This mattered because real users do not always enter perfect information.

Another technique I used was code coverage testing. Coverage showed how much of the code was reached by the tests. This was helpful because a project could have passing tests while still leaving important validation or service methods untested.

Some techniques I did not really use were integration testing, system testing, and acceptance testing. Integration testing would matter more if these services connected to a database, login system, or mobile app interface. System testing would matter once the full application was built. Acceptance testing would matter when the customer could use the finished app and decide whether it met their needs.

Each testing technique has a different purpose. Unit testing is useful early because it catches problems in individual classes. Integration testing becomes important when separate parts connect. System testing checks the full application before release, and acceptance testing helps confirm the product works for the customer. For this project, unit testing was the best fit because the services were small and class-based.

As a software tester, I tried to stay cautious. I did not want to assume the code worked just because it compiled. For example, a contact might have a null phone number, or an appointment might be created with a past date. I also had to think about how the classes connected. If the Contact class allowed bad data, then ContactService could store bad data too.

I tried to limit bias by testing against the written requirements instead of only testing what I expected to work. Since I wrote the code, it would be easy to assume my design was correct.

Using the requirements like a checklist helped reduce that. Bias can be a concern when developers test their own code because they may avoid edge cases without realizing it.

This project reminded me that small code mistakes can become bigger problems later if they are ignored. If I only tested the easy cases, defects could escape into later stages and create technical

debt. To avoid technical debt, I plan to keep writing focused unit tests, test invalid inputs, check coverage, and fix issues while they are still small. Testing is not just extra work after coding. It is part of writing better software.

References

JUnit Team. (n.d.). *JUnit 5 user guide*. <https://junit.org/junit5/docs/current/user-guide/>